

虽然一直希望写一个 IF 启动切闭环的小笔记，但市面上关于这方面的书籍并不多（我还是希望能找到一些专业书籍支持我的结论而不是自己编一种方法出来），而更多的是在各种文献（但是我懒得看文献）和网上视频小碎片中获得具体的方法，既然没有一个比较好的参考标准，那就只能自己根据各种碎片信息写一下了。

• IF 启动的作用

在某些观测器，比如 SMO 这种基于反电动势的观测器，在低速的时候，直接闭环是没办法启动的，因为反电动势太小了，被采样精度或者各种噪声淹没了，所以 SMO 根本就没办法获取准确的位置信息，因此可以先使用 IF 启动将它拖到一个中等的转速，SMO 能够估计到准确转速的时候切换闭环。

此外，低速性能好的 EKF 还有非线性磁链观测器，虽然说我总是能够测得它可以启动电机，但是我觉得它仍然有那么一点点的可能性（毕竟我只测试了一款电机和驱动板），会启动失败，所以说我觉得统一使用 IF 将它启动到中高速，然后切换闭环，是一种比较好的方法，毕竟可以使得启动率达到 100%完美程度。

• IF 启动开始（准备阶段）

IF 启动通常给定一个恒定的电流 I，然后将速度慢慢提升上去，对于表贴式永磁同步电机

$$T_e = \frac{3}{2} i_q P_n \psi_f$$

如果我们给定一个恒定的电流，如

$$i_d = 0A, i_q = 0.6A$$

根据公式我们可以知道的是，得到的转矩是恒定的（这句话有误，后面讲为什么）。

你要问怎么维持一个恒定的电流？事实上我们维持恒定的电压很简单，但是维持恒定的电流，就要依靠 i_d , i_q 电流环，事实上我们只需要简单的让 i_d 电流环的给定为 0， i_q 电流环的给定为 0.6，PI 控制器会自动会搞定这件事情（它会自动调节给定电压以维持电流恒定），我们不需要管这个事情。

选定一个电角加速度，例如 $\alpha = \frac{d\omega}{dt} = 100 \text{rad/s}^2$ ，那么我们给定的速度就是（假设我们的中断是 20kHz 即 5×10^{-5} 秒）

$$\omega += \alpha * T_s = 100 \text{rad/s}^2 * 5 * 10^{-5} \text{s} = 0.005 \text{rad/s}$$

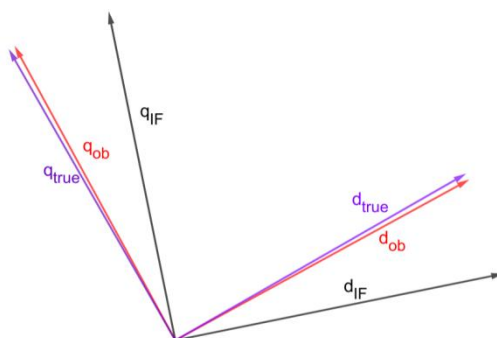
即程序里每次中断，给 ω 加 0.005 即可

当然，对应的角度是

$$\theta += \omega * T_s$$

• IF 启动过程（一阶段）

前面我们给定一个恒定的电流幅值，以及旋转的越来越快的电流相位，可以把电机拖起来，然而我们遇到了一个问题，强拖的角度和观测器的角度是不一样的，理论上，强拖角度会滞后于观测器角度（图示旋转方向逆时针，true 为转子实际位置，ob 为观测器位置）



事实上这就是我们之前讲的转矩的问题所在

$$T_e = \frac{3}{2} i_q P_n \psi_f$$

$$i_d = 0A, i_q = 0.6A$$

很明显，这里的 i_q 指的是电机转子实际位置在 q 轴的电流分量（图示 q_true ），但是我们肯定是不可能知道电机转子实际位置的，哪怕是我们的观测器的角度，也肯定是与电机转子实际位置有一点小偏差（是超前还是滞后我不太清楚但是偏差是比较小的），我们程序里使用IF启动的角度和电机转子实际位置角度事实上有一个很大的偏差，导致实际输出的转矩是

$$T_e = \frac{3}{2} i_{q_true} P_n \psi_f$$

$$i_{q_true} = i_{q_IF} \cos(\theta_{true} - \theta_{IF}) = 0.6 \cos(\theta_{true} - \theta_{IF})$$

我们做FOC获取转子位置，就是想要把所有的电流落到实处，都用来提供转矩（这里扯远了），但是IF强拖的时候，很明显我们注入0.6A所谓的 i_q 电流，得到的转矩并不是0.6A，而是比它要小，那么转矩到底是多少呢？事实上由负载决定，一般来说当电机转速越高，负载越大，因此当转速慢慢提高的时候， i_{q_true} 也慢慢提高，当然，电流幅值 $\sqrt{i_d^2 + i_q^2}$ 是不变的，只是 $\cos(\theta_{true} - \theta_{IF})$ 越来越接近1， i_{d_true} 的电流慢慢转到 i_{q_true} 上面罢了。很明显负载越大， $\cos(\theta_{true} - \theta_{IF})$ 越接近1，即 θ_{true} 越接近 θ_{IF}

我们通常使用大于负载所需的电流来启动电机，比如你要1000rad/s，并且IF强拖到500rad/s切闭环，并且你知道电机在500rad/s并且使用观测器闭环时，拖动负载需要的电流（通常是 i_q ）是0.3A，那么IF强拖启动的时候，你通常要给定一个大于0.3A的电流，你可能会说这样子太浪费了（电流根本没落到实处），但是这是必要的浪费，例如你抠抠搜搜给定一个0.3A的 i_q 用来强拖启动，前面小速度的时候还是风平浪静，但是后面当速度上来的时候，某一时刻由于负载的变化（比如突然大了那么一点），由于IF强拖时给定的是恒定的0.3A而不是速度环自动调节，那么在那一刻

$$i_{q_true} = 0.3 \cos(\theta_{true} - \theta_{IF}) < 0.3A$$

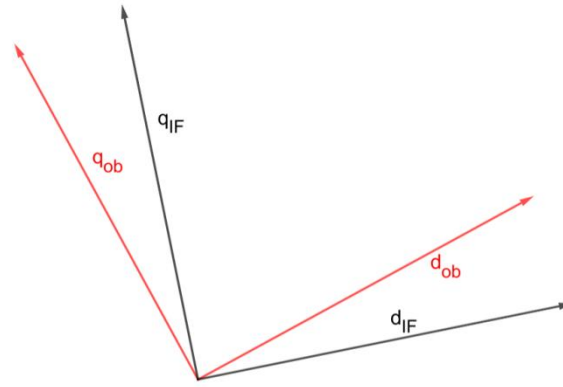
给定的转矩小于负载转矩，转子减速，但是IF启动仍然以那个角速度继续增加角度（甚至在IF启动时，角速度还会慢慢增加），于是转子实际角度越来越跟不上IF启动的角度，一下子就失步停机了。

总而言之，IF启动时，转矩由负载决定，而 $\theta_{true} - \theta_{IF}$ 也由负载决定，负载越大， θ_{true} 和 θ_{IF} 越接近，IF启动必须给定一个能够容纳最大负载情况的 i_{q_IF} ，留足一定的裕量。

• IF切闭环存在的问题（分析问题）

为了简化分析，我们忽略掉转子实际位置和观测器输出位置的误差，事实上也应当忽略，不仅仅是因为它们误差小的原因，而是我们根本不知道实际位置，并且最重要的一点是，切闭环是切到观测器角度而不是实际角度（你根本不知道实际角度，除非你有编码器，但是你有编码器为什么要用无感呢？）

我们的图片被简化为以下的情形（这里值得注意的是，在IF启动的过程中，我们也是开启观测器的，随着速度增加，观测器会慢慢收敛，我们在切闭环的时候，其实观测器已经准备就绪，收敛了）



我们计划将程序中使用的 θ_e ，从

$$\theta_e = \theta_{IF}$$

变成

$$\theta_e = \theta_{ob}$$

但是事实上我们不能直接切换，这是比较头疼的问题，首先我们 IF 启动到 500rad/s，已知程序的信息：

$I_q.set = 0.6A, I_q.measure = 0.6A, I_q.Output = \text{某个能够维持 } I_q = 0.6A \text{ 的数值}$

$I_d.set = 0A, I_d.measure = 0A, I_d.Output = \text{某个能够维持 } I_d = 0A \text{ 的数值}$

$Speed.set = 1000rad/s, Speed.measure = 500rad/s$

$Speed.Output = \text{无效数值，甚至由于长期达不到给定 } 1000rad/s, \text{ 数值已经非常大或者达到限幅}$

你就直接这样闭环，一闭一个不吱声，首先是角度

如果程序在 IF 启动一阶段结束（结束标志为 ω 达到 500rad/s），直接让 $\theta_e = \theta_{IF}$ ，那么这次中断或下次中断（取决于你程序怎么写），有

$I_q.set = 0.6A, I_q.measure = 0.6\cos(\theta_{ob} - \theta_{IF})A, I_q.Output = \text{之前的积分} + \text{比例误差项}$

$I_d.set = 0A, I_d.measure = 0.6\sin(\theta_{ob} - \theta_{IF})A, I_d.Output = \text{之前的积分} + \text{比例误差项}$

不管咋样，可以预见的是，比例误差项，即 $P*(set-measure)$ 将会变得很大或者说，Output 将会突然变得非常奇怪。电机会咯噔一下或者直接停机。

除此之外，转速环也在搞事情，如果 $I_q.set = Speed.Output$ ，即那个无效数值或者说奇奇怪怪的限幅值， $I_q.set$ 也会变得非常奇怪，同时 $I_q.Output$ 也会更加奇怪，因为 $P*(set-measure)$ 中的 set 和 measure 都变得奇怪了。

当然，根据

$$Speed.Output = P * (Set - Measure) + Addup * I$$

我们可以将转速环的积分项重置为

$$Addup = \frac{Speed.Output - P * (Set - Measure)}{I}$$

其中可以取

$$Speed.Output = I_q.set = 0.6A$$

这样做的好处是，至少这个中断或者说下个中断内， $I_q.set$ 不会变得非常奇怪了，但是之前说的 $I_q.Output$ 和 $I_d.Output$ 仍然会把这一切搞砸。

• IF 切闭环方法———加权过渡

好吧，其实我不打算讲这个方法，因为我试过实际上是很难搞，可能是我的代码写的不行，但是这个切换起来真的非常复杂，要考虑很多东西，写着写着各种切换策略把整个代码弄得一塌糊涂，逻辑混乱而不够优雅。

它的主要思想是，切换时：

$$\theta_e = k\theta_{ob} + (1 - k)\theta_{IF}$$

其中 k 随着时间慢慢地从 0 变为 1（当然这里要注意如果 θ_{ob} 和 θ_{IF} 被限制为 $0 \sim 2\pi$ 的时候，在边界处可能出现的问题，如 $k=0.5$ ，上一时刻 $\theta_{ob} = 2 * \pi - 0.05$ ， $\theta_{IF} = 2 * \pi - 0.1$ ， $\theta_e = 2 * \pi - 0.075$ ，一切正常，下一时刻可能 $\theta_{ob} = 0$ ， $\theta_{IF} = 2 * \pi - 0.05$ ， $\theta_e = \pi - 0.025$ ，很明显，这里被砍掉了一个 π ，是 θ_{ob} 从 2π 溢出到 0 导致的，这里需要写程序处理这种情况，但是不够优雅，所以不想搞了），而 θ_{IF} 此时应该是随着观测器的输出速度增加的（即 $\dot{\theta}_{IF} = \omega_{ob} T_s$ ）

这个加权过渡看起来似乎是非常的美妙：

$$I_q.set = 0.6A, I_q.measure = 0.6\cos(\theta_e - \theta_{IF})A, I_q.Output = \text{之前的积分} + \text{比例误差项}$$

$I_d.set = 0A, I_d.measure = 0.6\sin(\theta_e - \theta_{IF})A, I_d.Output = \text{之前的积分} + \text{比例误差项}$
这里 $\cos(\theta_e - \theta_{IF}) = \cos(k(\theta_{ob} - \theta_{IF}))$ ， $k=0$ 的时候，很明显 $I_q.measure = 0.6A$ ，因此 $I_q.Output$ 不会变化太大。然后随着 k 慢慢上升，再平滑过渡过去。
但是后面就容易失控了，前面说的 $I_q.set = 0.6A$ 是要比负载所需的电流要大的，因此如果过渡到 θ_{ob} ：

$$\theta_e = k\theta_{ob} + (1 - k)\theta_{IF} \rightarrow \theta_{ob}$$

那么 $I_q.set = 0.6A$ 要么把电机转的很快（或者说到达最大速度，电流环饱和都没能使得 $I_q=0.6A$ ），要么过渡中把电机弄停了。

此外我还是没能找到合适的方式把转速闭环。如果有人知道如何实现加权过渡法的可以告诉我一下。

• IF 切闭环方法———功角自平衡

这是一个不错的方法，前面我们说了，

$$T_e = \frac{3}{2}i_{q,true}P_n\psi_f$$

$$i_{q,true} = i_{q,IF}\cos(\theta_{true} - \theta_{IF}) \approx 0.6\cos(\theta_{ob} - \theta_{IF})$$

这个 $i_{q,true}$ 由负载决定，负载大 $i_{q,true}$ 就大，负载小 $i_{q,true}$ 就小，或者说负载大 θ_{ob} 就越接近 θ_{IF} ，负载小 θ_{ob} 就越超前 θ_{IF} 。

这时我们可以慢慢减小 IF 给定，即下式

$$i_{q,true} \approx 0.6k\cos(\theta_{ob} - \theta_{IF})$$

k 从 1 慢慢减小而若负载不变则 $\cos(\theta_{ob} - \theta_{IF})$ 慢慢增加，在某个时候， $0.6k$ 刚刚好就是负载需要的实际 i_q 电流，即 $\cos(\theta_{ob} - \theta_{IF}) = 1$ ， $\theta_{ob} = \theta_{IF}$ 。此时各个 π 控制器的情况如下：

$$I_q.set = 0.6kA, I_q.measure = 0.6kA, I_q.Output = \text{某个能够维持} I_q = 0.6kA \text{的数值}$$

$$I_d.set = 0A, I_d.measure = 0A, I_d.Output = \text{某个能够维持} I_d = 0A \text{的数值}$$

$$Speed.set = 1000\text{rad/s}, Speed.measure = 500\text{rad/s}$$

$$Speed.Output = \text{无效数值，甚至由于长期达不到给定} 1000\text{rad/s，数值已经非常大或者达到限幅}$$

在切换闭环时，除了让程序中使用的电角度从 IF 转成观测器即 $\theta_e = \theta_{ob}$ ，以外，我们程序中的 $I_q.set$ 也要赶紧转到 $Speed.Output$ 中了（即维持 $I_q.set = Speed.Output$ ），当然

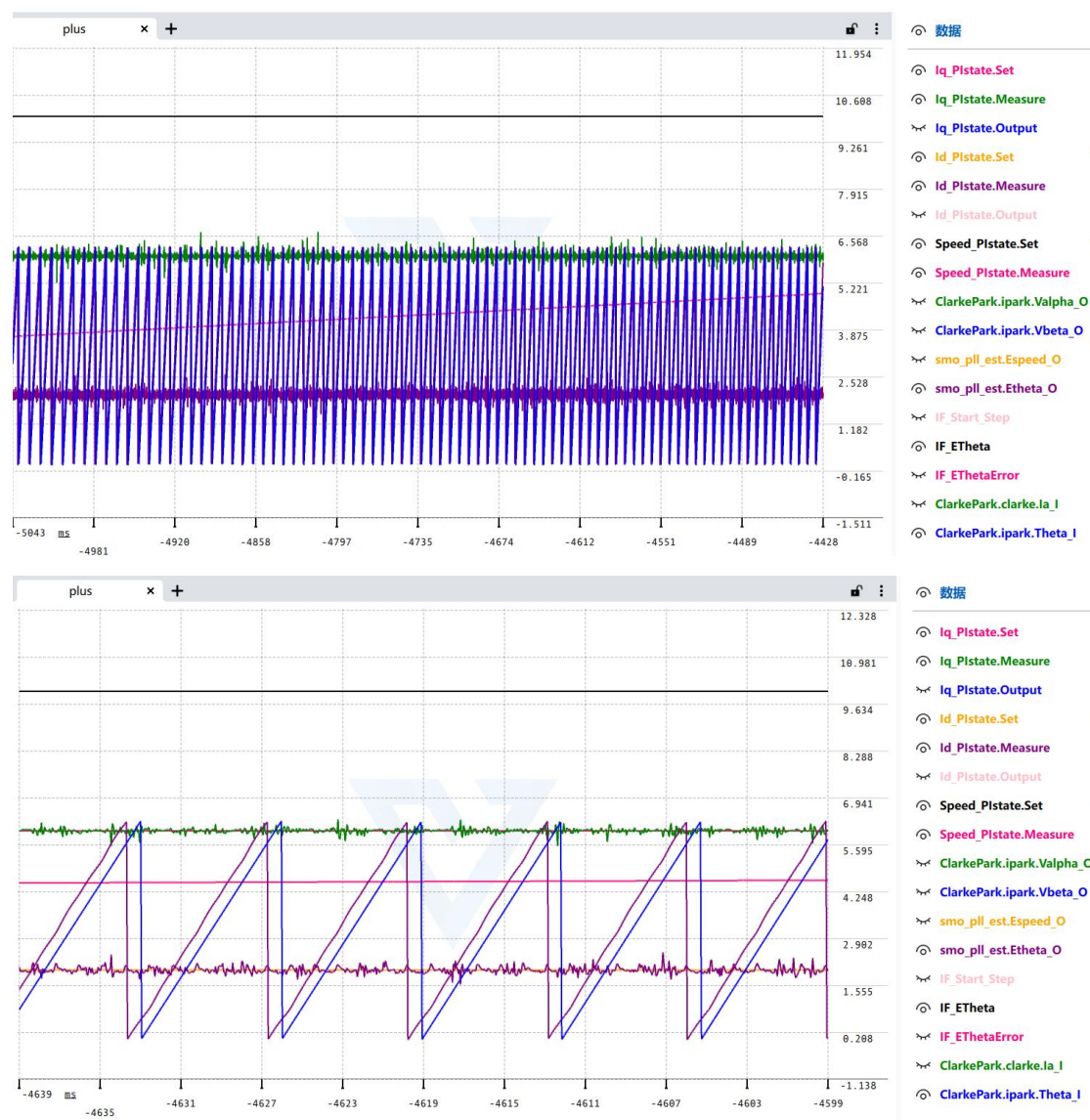
*Speed.Output*现在还是无效数值，我们应该使用这个式子计算：

$$Addup = \frac{Speed.Output - P * (Set - Measure)}{I}$$

这是前面讲过的式子，这样可以使得转速环的输出刚刚好是 0.6k，切换瞬间，*Iq.Set* 没有跳变，还是 0.6k。

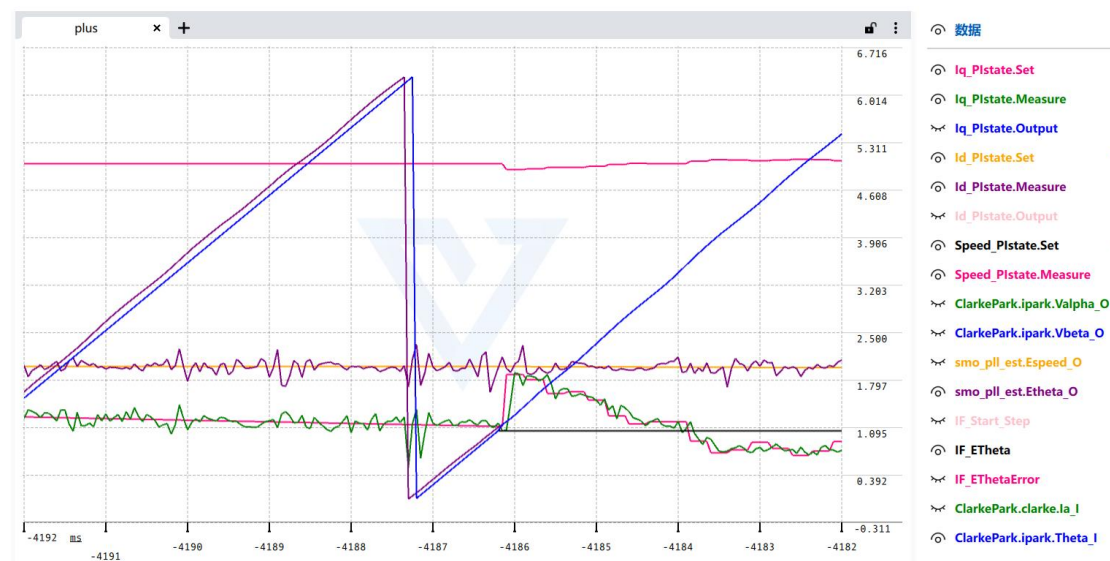
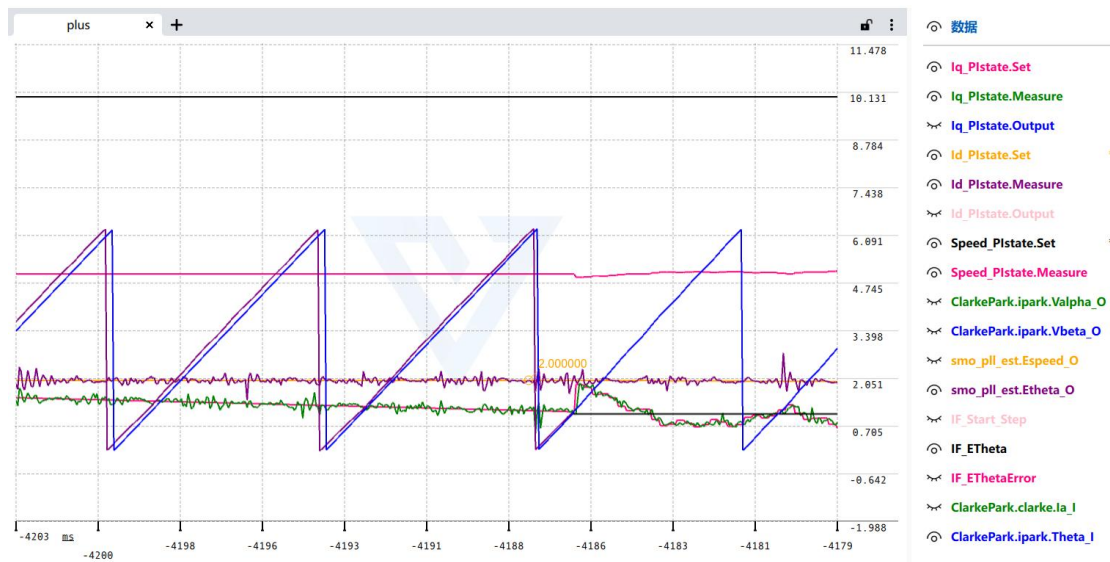
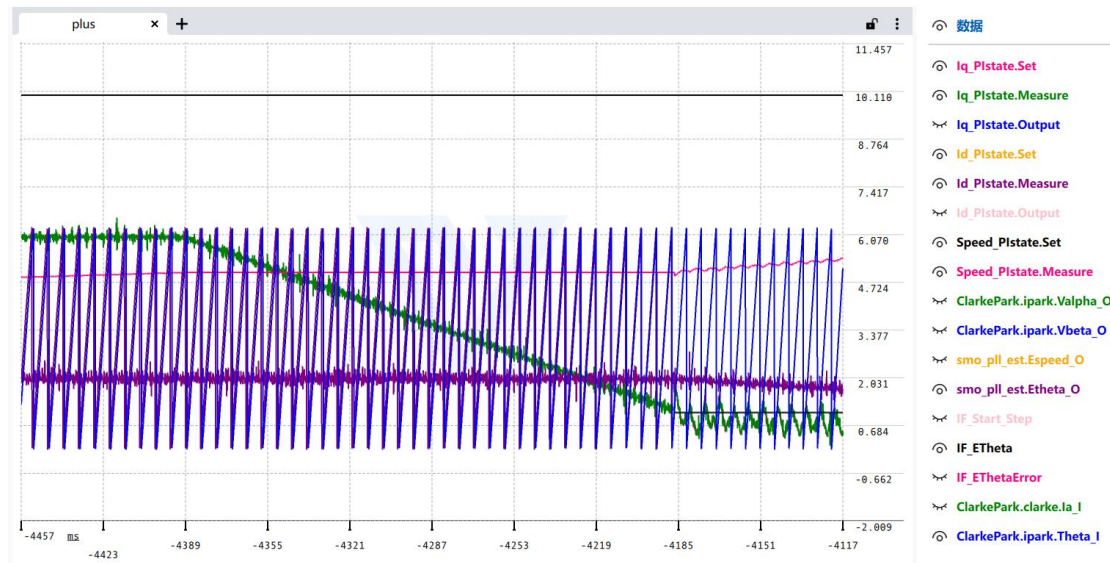
实际测试时，我们使用附录 1 的代码（这个代码由于不是我的最终方案，所以只写了 SMO 的强拖，而不是把全部观测器都强拖），在空载下测试启动，为了在切换时没有顿挫感，还增加了 *id=0.2A* 的注入

强拖过程如下两张图所示，为了能在一张图上展示速度、电流以及角度，对电流进行 10 倍放缩（即你看到 *Iq_Plstate.Set=6* 实际上是 *Iq_Plstate.Set=0.6*），转速进行 0.01 倍放缩



从上图可以看到，在强拖一阶段时，角度即 *ClarkePark.ipark.Theta_I* 使用了 *IF_ETheta*（这也是为什么图中看不到 *IF_ETheta* 的原因，因为它被 *ClarkePark.ipark.Theta_I* 完全遮住了），然后 *Iq* 给了 0.6A 而 *Id* 给了 0.2A（看得出来电流环跟随的很好），速度慢慢上去（粉色），而观测器 *smo_pll_est.Etheta_O* 则在意料之中地滞后于强拖角度。

如下三图，强拖二阶段时（即转速达到我们的阈值 500rad/s），缓慢降低 Iq 给定（即被绿色线遮住的粉色 Iq_Plstate.Set），smo_pll_set.Etheta_O 如意料之中地慢慢追上 IF_Etheta



如上三图，在空载情况下， I_q 降到了接近 0.1（注意 I_q 放大了 10 倍，在图上看到是 1）的时候， $\text{smo_pll_set.Etheta_O}$ 靠近了 IF_theta ，于是将 $\text{ClarkePark.ipark.Theta_I}$ 切换为 $\text{smo_pll_set.Etheta_O}$ ，即切换闭环，此时不再更新 IF_theta ，已经不需要它了（图中平行的一条黑线表示它没有被更新）。

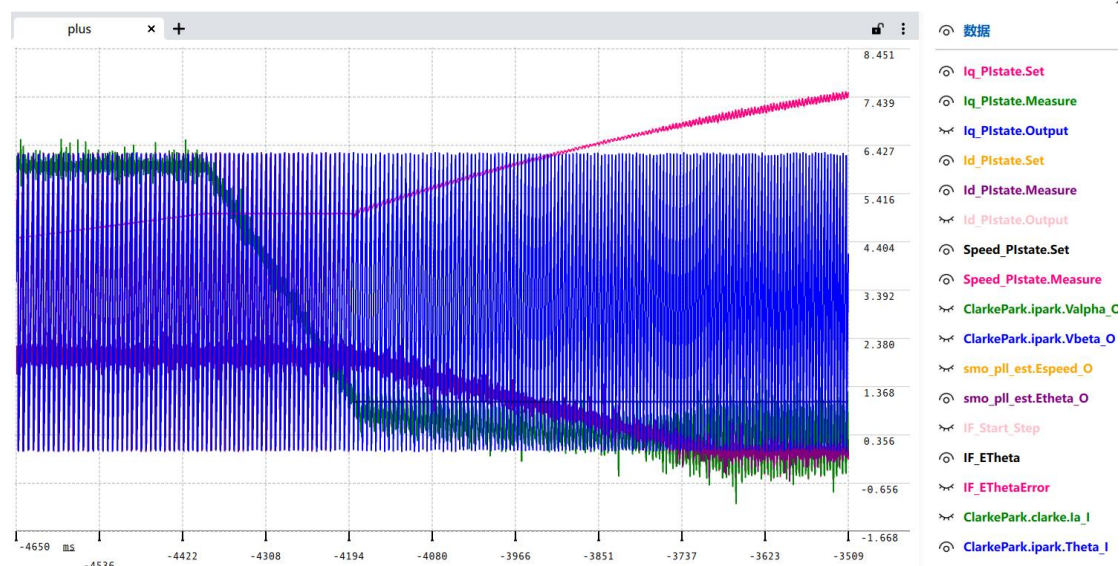
在这个过程中，我们使用

$$\text{Addup} = \frac{\text{Speed.Output} - P * (\text{Set} - \text{Measure})}{I}$$

计算转速环的积分项，按理说 $I_q.\text{Set}$ 不会在切换时突变，但是其实 $I_q.\text{Set}$ 实际上还会突跳到接近 0.17A 的情况。事实上可以看到 Speed.Measure 在切换的时候下降了一点，因此多出来的一点 0.07A 应该是转速 Measure 降落，然后 $P * (\text{Set} - \text{Measure})$ 增大的结果。（说明： Measure 闭环前为强拖 500rad/s，闭环后， Measure 为 SMO 的输出 491rad/s，而转速环 P 为 0.008712368f，则 $I_q.\text{Set}$ 增大 $0.008712368f * (500 - 491) = 0.078A$ ）

整体而言，在测试时，切换闭环，电机出现一点小抖动和顿挫感，但不太明显。

收尾阶段如下图所示，就是 I_d 给定慢慢降低（紫色线和被遮住的黄色线），然后由于闭环了，观测器闭环，转速环也接管了 $I_q.\text{set}$ ，于是转速慢慢上升，直到给定的 1000rad/s



功角自平衡法代码如附录 1 所示（翻到文档最后面），由于不是本程序使用的方法，所以只写了对 SMO 进行强拖的代码。

• IF 切闭环方法——坐标变换法

这个是我自己取的名字。事实上确实是有论文讲解相关的方法（可在 IEEE 网站搜：Seamless Start-up of Loaded PMSM Drive from Open-Loop Id/F to Sensorless FOC），并且将它称之为无缝切换（Seamless Start-up）方法，但是我却懒得去看论文，只扫了一眼。

坐标变换法的原理：

前面我们说了，将程序中的 θ_e 硬生生从 θ_{IF} 切换为 θ_{ob} ，那么下一刻电流环的各项信息是这样的：

$$I_q.\text{set} = 0.6A, I_q.\text{measure} = 0.6\cos(\theta_{ob} - \theta_{IF})A, I_q.\text{Output} = \text{之前的积分} + \text{比例误差项}$$

$$I_d.\text{set} = 0A, I_d.\text{measure} = 0.6\sin(\theta_{ob} - \theta_{IF})A, I_d.\text{Output} = \text{之前的积分} + \text{比例误差项}$$

这时， $I_q.Output$ 和 $I_d.Output$ 都有大幅的跳变，因为 $I_q.set = 0.6A \neq I_q.measure = 0.6\cos(\theta_{ob} - \theta_{IF})$ ，比例项将使得 Output 突然变大/变小。

转换一下思路，要是在切换前，我们将 Set 改为 measure 的值呢？

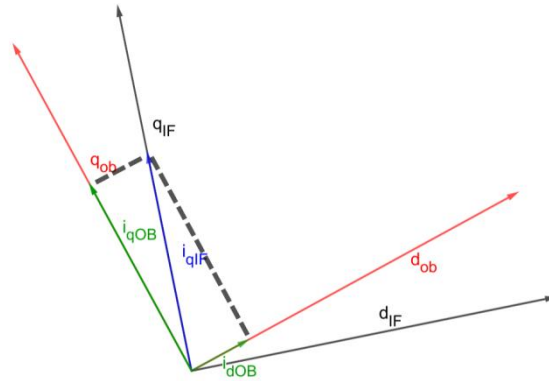
$$I_q.set = 0.6\cos(\theta_{ob} - \theta_{IF})A, I_q.measure = 0.6\cos(\theta_{ob} - \theta_{IF})A$$

$$I_q.Output = \text{之前的积分} + \text{比例误差项}$$

$$I_d.set = 0.6\sin(\theta_{ob} - \theta_{IF})A, I_d.measure = 0.6\sin(\theta_{ob} - \theta_{IF})A$$

$$I_d.Output = \text{之前的积分} + \text{比例误差项}$$

这样子 $I_q.set = 0.6\cos(\theta_{ob} - \theta_{IF}) = I_q.measure = 0.6\cos(\theta_{ob} - \theta_{IF})$ ，比例项就不会突然变大，输出也不会跳变了。



上面的图片形象地说明了这一切，强拖时， $i_{qIf}=0.6A$ ，而 $i_{dIf}=0$ 我们就不画出来了，切换闭环的瞬间，将 $i_q.set$ 和 $i_d.set$ 的值都设为 i_{qIf} 对观测器坐标系(d_{ob}, q_{ob})的投影值，然后把角度直接切成观测器角度即可，保证了 Set 和 Measure 是一样的。可以写成以下形式：

$$\begin{bmatrix} I_{d_{ob}.set} \\ I_{q_{ob}.set} \end{bmatrix} = \begin{bmatrix} 0.6\sin(\theta_{ob} - \theta_{IF}) \\ 0.6\cos(\theta_{ob} - \theta_{IF}) \end{bmatrix}$$

当然，你说 $i_{dIf}=0$ ，那么不为零的情况下呢？这就是非常有趣的 Park 变换了：

$$\begin{bmatrix} I_{d_{ob}.set} \\ I_{q_{ob}.set} \end{bmatrix} = \begin{bmatrix} \cos(\theta_{ob} - \theta_{IF}) & \sin(\theta_{ob} - \theta_{IF}) \\ -\sin(\theta_{ob} - \theta_{IF}) & \cos(\theta_{ob} - \theta_{IF}) \end{bmatrix} \begin{bmatrix} I_{d_{IF}.set} \\ I_{q_{IF}.set} \end{bmatrix}$$

我们可以直接使用现有的 Park 变换代码实现这一切，而不需要自己写别的什么东西，当然，代码里面的 Park 变换结构是这样的：

```
struct Park_t {
    float lalpha_I;
    float lbeta_I;

    float Theta_I; // 弧度制

    float Id_O;
    float Iq_O;
}park;
```

其中 $lalpha_I = I_{d_IF}.set$ ， $lbeta_I = I_{q_IF}.set$ ， $Theta_I = \theta_{ob} - \theta_{IF}$ ， $Id_ob.set = Id_O$ ， $Iq_ob.set = Iq_O$ ，好吧，我承认这个代码中的 Park 变换的结构取名可能不太直观和对应，毕竟谁会想到把 I_{d_IF} 赋值到 $lalpha_I$ 这两个命名根本不一样的变量呢（ d 赋值到 $alpha$ ，真是让人摸不着头脑啊！），当然后面还有更多不直观的。

话说我们将电流环的 Set 设定和 Measure 一样了，这就意味着 Output 不会在切换的时候大幅度跳变，但是事实上 Output 仍然会大幅度跳变，因为我们生成的 U_d, U_q 原本使用 θ_{IF} 转换

为 U_{α} , U_{β} 电压，然而切换瞬间，却使用 θ_{ob} ，所以实际上的 U_{α} , U_{β} 电压仍然是跳变的。

解决方案仍然是做一次 Park 变换：

$$\begin{bmatrix} I_{d_{ob}}.Output \\ I_{q_{ob}}.Output \end{bmatrix} = \begin{bmatrix} \cos(\theta_{ob} - \theta_{IF}) & \sin(\theta_{ob} - \theta_{IF}) \\ -\sin(\theta_{ob} - \theta_{IF}) & \cos(\theta_{ob} - \theta_{IF}) \end{bmatrix} \begin{bmatrix} I_{d_{IF}}.Output \\ I_{q_{IF}}.Output \end{bmatrix}$$

这里如果使用 Park 变换，就是 $I_{\alpha} = I_{d_{IF}}.Output$ ， $I_{\beta} = I_{q_{IF}}.Output$ ， $\theta = \theta_{ob} - \theta_{IF}$ ， $I_{d_{ob}}.Output = I_{d_O}$ ， $I_{q_{ob}}.Output = I_{q_O}$ ，可以说这个命名确实不太对应。

我们进行一次 Park 变换得到的 Output 事实上是不够的，在下次计算时，Output 又会变为原来的样子，所以我们需要重置积分项，前面在转速环的时候已经说过了，也就不再推导了，把它写出来就算了：

$$Addup = \frac{I_{d_Output} - P * (I_{d_Set} - I_{d_Measure})}{I}$$

当然，由于电流环总是能够把 Set 和 Measure 跟踪的很好 ($Set \approx Measure$)，也就是上式可以简化为：

$$Addup = \frac{I_{d_Output}}{I}$$

接下来更新一下转速环：

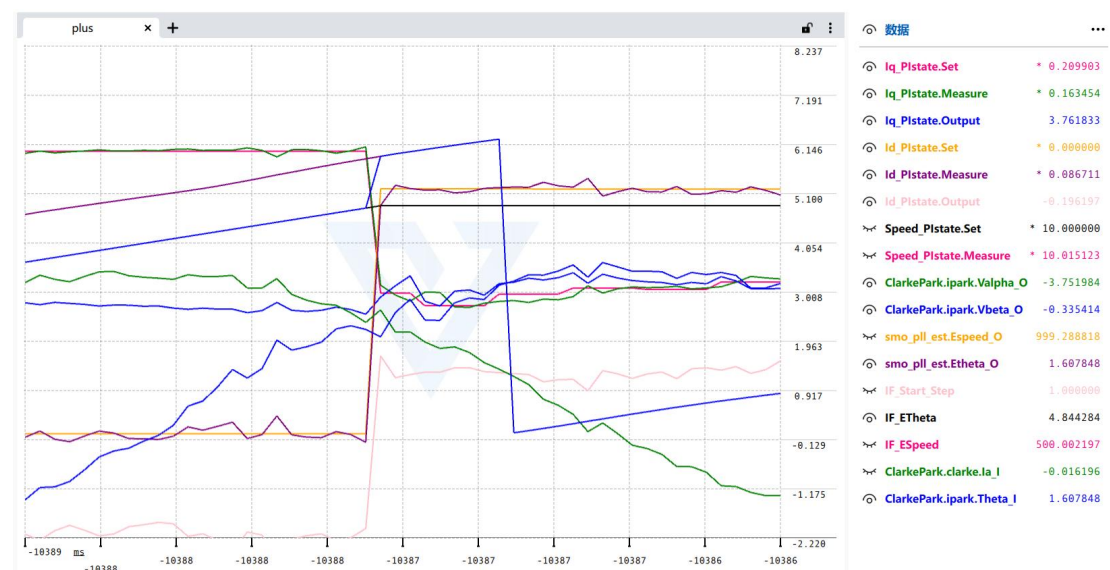
$$Speed_Addup = \frac{Speed_Output - P * (Set - Measure)}{I} = \frac{I_{q_Set} - P * (Set - Measure)}{I}$$

这样子能够保证转速环更新的时候， I_{q_Set} 不会突变。

至此，我们处理完毕就可以将 θ_e 切换为 θ_{ob} 了。

然后由于一般我们使用 $I_{d_Set} = 0$ 进行控制，而通过强拖，Park 变换后， I_{d_Set} 通常不为零（通常是一个正值），所以最后还要进行收尾工作，将 I_{d_Set} 慢慢减小到零。

实际运行效果如下图所示



切换的过程只有一瞬间，所以我们只需要截这一刻的图就可以表达整个切换过程，当然你可能会破口大骂，这哪跟哪？那么多波形混在一起怎么看得清楚？别急，我们逐一分析。

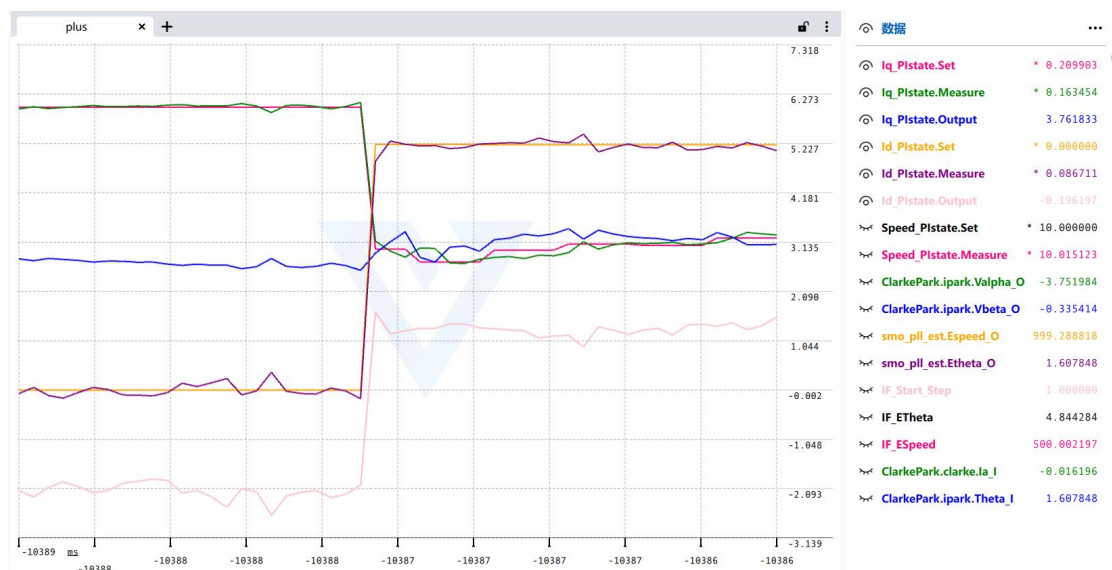
首先是角度，如下图所示



很明显这个角度和我们分析的一模一样，IF 角度确实是要滞后于 SMO 角度的，而切换前程序使用的角度（ClarkeParke.ipark.Theta_I 可以看出来我们程序使用是什么角度）是 IF 角度，也就是蓝色线完美遮住了黑色线的那个部分。

切换后，IF 不再增加，而 ClarkeParke.ipark.Theta_I 突变为 SMO 角度。当然我们这种比较优秀的方法，强行更改角度的情况下，都能保持输出电压的连续性（后面会分析），也根本感受不到电机任何的顿挫感。这也是为什么程序最终选定了这种方法。

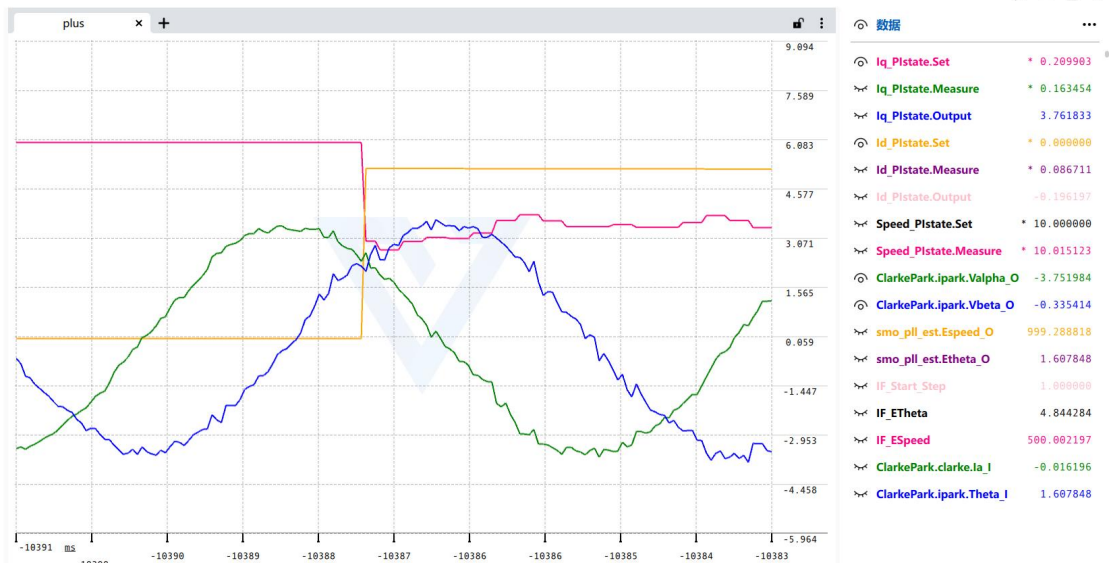
扯远了，我们继续看电流环的 Set 和 Measure 以及输出（电流环图示经过了 10 倍放大，图示 Iq.Set 是 6A 实际上是 0.6A）。



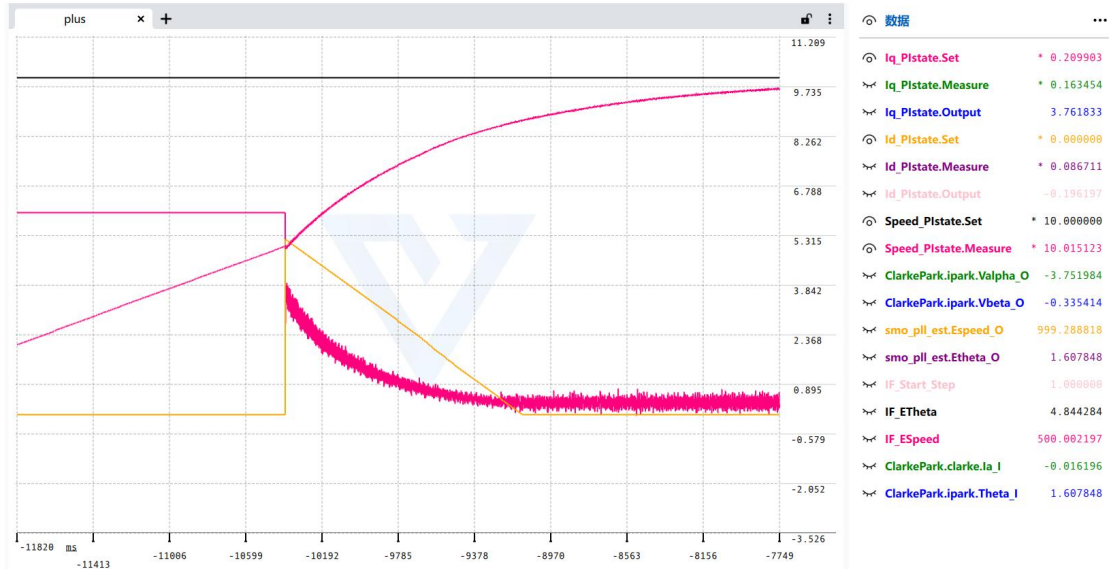
电流环在切换时，Iq.Set 从 0.6A 瞬间下降为 0.3A，而 Id.Set 则从 0 突变为 0.5A 左右，很明显经过坐标变换后，0.6A 的 Iq 强拖电流很大一部分被分到了 Id 分量里（这也是为什么我们必须使用准确角度进行 FOC 的原因，不然大部分电流都不用来产生转矩而是浪费在 d 轴上，不过也是扯远了），而 Set 和 Measure 也是保持一致的，没什么变化，这也是意料之内的。Output 也是一样的，d 轴的 Output 从 -2V 突变为一个正值 1V，而 q 轴的 Output 则从 2.5V 跳变为 3V（注意，Vq 的增大并不是一件奇怪的事情，因为 Vd_IF 经过坐标变换后，有相当一部分转移到 Vq_OB 了）。

下图展示了 Valpha 和 Vbeta 的变化，可以看到，我们切换了坐标系，而实际生成的 Valpha

和 V_{β} 是连续的，这就是这套方案的优点，生成的电压是连续的，没有突变。



最后总体收尾如下图所示，将 I_d 的给定慢慢减小到零，而转速也是慢慢上去了：



• 电流环前馈解耦存在的问题

我们把电流环前馈解耦关闭，也就是在“s 域分析与参数整定.pdf”中说明的

$$\begin{cases} u_d = u_{d,PI} - \omega_e L_q i_q \\ u_q = u_{q,PI} + \omega_e (L_d i_d + \psi_f) \end{cases}$$

这里若开启电流环前馈解耦，则 $u_d \neq u_{d,PI}$ ，我们的坐标变换切闭环还要考虑真实的 u_d 项，为了代码简洁，把电流环前馈解耦关闭，即

$$\begin{cases} u_d = u_{d,PI} \\ u_q = u_{q,PI} \end{cases}$$

上述式子成立，那么我们就可以使用前面所说的坐标变换切闭环。事实上，推导出支持前馈解耦的坐标变换切闭环方法应该不难，如果有时间我会把这部分补上。

• 附录 1：功角自平衡代码

```
/* USER CODE BEGIN Header */

/**
 * *****************************************************************************
 *
 * @file    stm32g4xx_it.c
 *
 * @brief   Interrupt Service Routines.
 *
 * *****************************************************************************
 *
 * @attention
 *
 *
 * Copyright (c) 2025 STMicroelectronics.
 *
 * All rights reserved.
 *
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 *
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 *
 * *****************************************************************************
 */

/* USER CODE END Header */

/* Includes -----*/

#include "main.h"
#include "stm32g4xx_it.h"

/* Private includes -----*/

/* USER CODE BEGIN Includes */

#include "USB_JustFloat.h"
#include "PMSM_Control_Core/Clarke_Park.h"
#include "PMSM_Control_Core/EKF.h"
#include "PMSM_Control_Core/Hardware.h"
#include "PMSM_Control_Core/PI_Controller.h"
#include "PMSM_Control_Core/SVPWM.h"
#include "PMSM_Control_Core/FluxObserver_PLL.h"
#include "PMSM_Control_Core/SMO_PLL.h"
#include "PMSM_Control_Core/ST-SMO_PLL.h"
#include "PMSM_Control_Core/FluxWeakening.h"

/* USER CODE END Includes */

/* Private typedef -----*/

/* USER CODE BEGIN TD */

/* USER CODE END TD */

/* Private define -----*/

/* USER CODE BEGIN PD */
```

```

/* USER CODE END PD */

/* Private macro -----*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables -----*/
/* USER CODE BEGIN PV */
/*
* IF 强拖切闭环步骤
* 数字0:IF 强拖步骤,速度不停增加(IdIq 电流环开启,转速环关闭,观测器开启,角度:由IF 给定)
* 数字1:IF 强拖结束,减小Iq 给定直到观测器角度与强拖角度靠近(IdIq 电流环开启,转速环关闭,观测器开启,角度:由IF 给定)
* 数字2:切换闭环(IdIq 电流环开启,转速环开启,观测器开启,角度:由观测器给定)
*/
uint8_t IF_Start_Step=0;
static const float IF_IqCurrentTarget=0.6f; // IF 强拖电流最终给定(给定 Iq 电流环最终值,单位 A)
static const float IF_IqCurrentAcceleration=0.5f; // IF 强拖电流增加速度(A/s)
float IF_IqCurrent=0.0f; // IF 强拖电流实际值(Iq 给定值以每秒 IF_IqCurrentAcceleration 的速度增加,慢慢上升直到
IF_IqCurrentTarget)
static const float IF_Acceleration=100.f; // IF 加速度(表示强拖速度增加的速度,单位 rad/s^2)
static const float IF_Target_Speed=500.f; // IF 目标速度(表示结束强拖应该达到的速度,即 IF_Start_Step 何时变为 1 的阈值,
单位 rad/s)
float IF_ETheta=0.0f; // IF 角度
float IF_ESpeed=0.0f; // IF 速度
static float IF_Weight=0; // IF 二阶段权重数值
static const float IF_Weight_Total=10000; // 总权重,缓慢降低 Iq 给定直到 IF 角度超前量慢慢减小并与 SMO 角度贴合
/* USER CODE END PV */

/* Private function prototypes -----*/
/* USER CODE BEGIN PFP */

/* USER CODE END PFP */

/* Private user code -----*/
/* USER CODE BEGIN 0 */
/* USER CODE END 0 */

/* External variables -----*/
extern PCD_HandleTypeDef hpcd_USB_FS;
extern DMA_HandleTypeDef hdma_adc1;
extern ADC_HandleTypeDef hadc1;
extern ADC_HandleTypeDef hadc2;

```



```

/* USER CODE BEGIN EV */

/* USER CODE END EV */

/*****
/*          Cortex-M4 Processor Interruption and Exception Handlers          */
/*****
/**
 * @brief This function handles Non maskable interrupt.
 */
void NMI_Handler(void)
{
    /* USER CODE BEGIN NonMaskableInt_IRQn 0 */

    /* USER CODE END NonMaskableInt_IRQn 0 */
    /* USER CODE BEGIN NonMaskableInt_IRQn 1 */
    while (1)
    {
    }
    /* USER CODE END NonMaskableInt_IRQn 1 */
}

/**
 * @brief This function handles Hard fault interrupt.
 */
void HardFault_Handler(void)
{
    /* USER CODE BEGIN HardFault_IRQn 0 */

    /* USER CODE END HardFault_IRQn 0 */
    while (1)
    {
        /* USER CODE BEGIN W1_HardFault_IRQn 0 */
        /* USER CODE END W1_HardFault_IRQn 0 */
    }
}

/**
 * @brief This function handles Memory management fault.
 */
void MemManage_Handler(void)
{
    /* USER CODE BEGIN MemoryManagement_IRQn 0 */

```

```

/* USER CODE END MemoryManagement_IRQn 0 */
while (1)
{
    /* USER CODE BEGIN W1_MemoryManagement_IRQn 0 */
    /* USER CODE END W1_MemoryManagement_IRQn 0 */
}

/**
 * @brief This function handles Prefetch fault, memory access fault.
 */
void BusFault_Handler(void)
{
    /* USER CODE BEGIN BusFault_IRQn 0 */

    /* USER CODE END BusFault_IRQn 0 */
    while (1)
    {
        /* USER CODE BEGIN W1_BusFault_IRQn 0 */
        /* USER CODE END W1_BusFault_IRQn 0 */
    }
}

/**
 * @brief This function handles Undefined instruction or illegal state.
 */
void UsageFault_Handler(void)
{
    /* USER CODE BEGIN UsageFault_IRQn 0 */

    /* USER CODE END UsageFault_IRQn 0 */
    while (1)
    {
        /* USER CODE BEGIN W1_UsageFault_IRQn 0 */
        /* USER CODE END W1_UsageFault_IRQn 0 */
    }
}

/**
 * @brief This function handles System service call via SWI instruction.
 */
void SVC_Handler(void)
{
    /* USER CODE BEGIN SVC_Call_IRQn 0 */

```

```

/* USER CODE END SVCall_IRQn 0 */

/* USER CODE BEGIN SVCall_IRQn 1 */

/* USER CODE END SVCall_IRQn 1 */
}

/**
 * @brief This function handles Debug monitor.
 */
void DebugMon_Handler(void)
{
/* USER CODE BEGIN DebugMonitor_IRQn 0 */

/* USER CODE END DebugMonitor_IRQn 0 */
/* USER CODE BEGIN DebugMonitor_IRQn 1 */

/* USER CODE END DebugMonitor_IRQn 1 */
}

/**
 * @brief This function handles Pendable request for system service.
 */
void PendSV_Handler(void)
{
/* USER CODE BEGIN PendSV_IRQn 0 */

/* USER CODE END PendSV_IRQn 0 */
/* USER CODE BEGIN PendSV_IRQn 1 */

/* USER CODE END PendSV_IRQn 1 */
}

/**
 * @brief This function handles System tick timer.
 */
void SysTick_Handler(void)
{
/* USER CODE BEGIN SysTick_IRQn 0 */

/* USER CODE END SysTick_IRQn 0 */
HAL_IncTick();
/* USER CODE BEGIN SysTick_IRQn 1 */

```

```

    /* USER CODE END SysTick_IRQn 1 */
}

/*****

/* STM32G4xx Peripheral Interrupt Handlers
*/
/* Add here the Interrupt Handlers for the used peripherals.
*/
/* For the available peripheral interrupt handler names,
*/
/* please refer to the startup file (startup_stm32g4xx.s).
*/
*****/

/**
 * @brief This function handles DMA1 channel1 global interrupt.
 */
void DMA1_Channel1_IRQHandler(void)
{
    /* USER CODE BEGIN DMA1_Channel1_IRQn 0 */

    /* USER CODE END DMA1_Channel1_IRQn 0 */
    HAL_DMA_IRQHandler(&hdma_adc1);
    /* USER CODE BEGIN DMA1_Channel1_IRQn 1 */

    /* USER CODE END DMA1_Channel1_IRQn 1 */
}

/**
 * @brief This function handles ADC1 and ADC2 global interrupt.
 */
void ADC1_2_IRQHandler(void)
{
    /* USER CODE BEGIN ADC1_2_IRQn 0 */
    static volatile uint8_t adc1_done = 0;
    static volatile uint8_t adc2_done = 0;
    if (__HAL_ADC_GET_FLAG(&hadc1, ADC_FLAG_JEOS))
    {
        __HAL_ADC_CLEAR_FLAG(&hadc1, ADC_FLAG_JEOS);
        adc1_done = 1;
    }
    if (__HAL_ADC_GET_FLAG(&hadc2, ADC_FLAG_JEOS))
    {
        __HAL_ADC_CLEAR_FLAG(&hadc2, ADC_FLAG_JEOS);
        adc2_done = 1;
    }
    /* USER CODE END ADC1_2_IRQn 0 */
    HAL_ADC_IRQHandler(&hadc1);

```

```

HAL_ADC_IRQHandler(&hadc2);

/* USER CODE BEGIN ADC1_2_IRQn 1 */

//HRTIM 每次更新时间同时触发ADC1,ADC2 采样 Ia,Ib,因此此中断一个周期触发两次,必须要等两路电流都采集完,才进行
控制

if (adc1_done && adc2_done)
{
    adc1_done = 0;
    adc2_done = 0;
}
else return;

/*
* 获取ABC 相电流
*/

const uint32_t va_source=HAL_ADCEx_InjectedGetValue(&hadc1, ADC_INJECTED_RANK_1);
const uint32_t vb_source=HAL_ADCEx_InjectedGetValue(&hadc2, ADC_INJECTED_RANK_1);
// 必须显式强转有符号整数, 否则结果会被隐式转化为无符号整数
const float Ia_raw=(IA_REF-(int32_t)va_source)*VCC_3V3/IA_K/4095.f;
const float Ib_raw=(IB_REF-(int32_t)vb_source)*VCC_3V3/IB_K/4095.f;

// 低通滤波去毛刺 虽然中值滤波能很好去除电流采样毛刺,但是会恶化控制效果,这里使用低通滤波代替了
//const float Ia = median_filter_Ia_5(Ia_raw);
//const float Ib = median_filter_Ib_5(Ib_raw);
//const float Ia=IIR_filter2A(Ia_median);
//const float Ib=IIR_filter2B(Ib_median);
//const float Ia = Ia_raw;
//const float Ib = Ib_raw;
const float Ia =lowPass_filter_Ia(Ia_raw);
const float Ib =lowPass_filter_Ib(Ib_raw);
const float Ic=-(Ia+Ib);

/*
* 执行一次 Clarke , 获取静止三相电流
*/

ClarkePark.clarke.Ia_I=Ia;
ClarkePark.clarke.Ib_I=Ib;
ClarkePark.clarke.Ic_I=Ic;
Clarke_transform(&ClarkePark.clarke);

const int Observer=2;// 0 表示使用EKF,1 表示使用FluxObserver-PLL,2 表示使用SMO-PLL,3 表示使用ST-SMO_PLL
float Espeed=0;
float Etheta=0;
static float Valpha_last=0.0f;

```



```

static float Vbeta_last=0.0f;

/*
* 注意这里输入电压取上一次中断时计算的电压,事实上应当取上上次中断时计算的电压
* 时刻图:
* 中断时刻0 周期0 中断时刻1 周期1 中断时刻2 周期2
* 中断时刻0 计算的电压(此时已经进入周期0),将会在周期1生效
* 所以说我们在中断时刻2 应当取作用在周期1 的电压
* 即 中断时刻0 计算的电压
* 但是我们的 ClarkePark.ipark 变量只记录了前一个周期即中断时刻1 计算的电压
* 我们使用 ClarkePark.ipark 的电压会有偏差
* 所以我们使用了 Valpha_last 和 Vbeta_last 记录了上上次计算的电压,更为精确
* 中断时刻0 ipark 计算前,记录 Valpha_last 和 Vbeta_last=ipark(即中断时刻1 的ipark),计算 ipark(中断时刻0 的ipark)
* 中断时刻1, ipark 计算前,记录 Valpha_last 和 Vbeta_last=ipark(即中断时刻0 的ipark),
* 中断时刻2,执行观测器,使用 Valpha_last 和 Vbeta_last(即中断时刻0 的ipark),
*/

if (Observer==0) {
    /*
    * 执行一次 EKF,获取转子角度和速度
    */
    IF_Start_Step=2; // 不使用强拖
    ekf_est.Ialpha_I=ClarkePark.clarke.Ialpha_O;
    ekf_est.Ibeta_I=ClarkePark.clarke.Ibeta_O;
    ekf_est.Valpha_I=Valpha_last;
    ekf_est.Vbeta_I=Vbeta_last;
    EKF_update(&ekf_est);
    Espeed=ekf_est.Espeed_O;
    Etheta=ekf_est.Etheta_O;
}
else if (Observer==1){
    /*
    * 执行一次 FluxObserver-PLL,获取转子角度和速度
    */
    IF_Start_Step=2; // 不使用强拖
    fluxObserver_pll_est.Ialpha_I=ClarkePark.clarke.Ialpha_O;
    fluxObserver_pll_est.Ibeta_I=ClarkePark.clarke.Ibeta_O;
    fluxObserver_pll_est.Valpha_I=Valpha_last;
    fluxObserver_pll_est.Vbeta_I=Vbeta_last;
    FluxObserver_PLL_update(&fluxObserver_pll_est);
    Espeed=fluxObserver_pll_est.Espeed_O;
    Etheta=fluxObserver_pll_est.Etheta_O;
}
else if (Observer==2){
    /*
    * 执行一次 SMO-PLL,获取转子角度和速度,注意 SMO 低速性能差,这里先使用磁链观测器把速度提上来再切换到 SMO

```

```

    */

    smo_pll_est.Ialpha_I=ClarkePark.clarke.Ialpha_O;
    smo_pll_est.Ibeta_I=ClarkePark.clarke.Ibeta_O;
    smo_pll_est.Valpha_I=Valpha_last;
    smo_pll_est.Vbeta_I=Vbeta_last;
    SMO_PLL_update(&smo_pll_est);
    if (IF_Start_Step==0) {
        IF_ESpeed+=IF_Acceleration*T_s;
        IF_ETheta+=IF_ESpeed*T_s;
        while (IF_ETheta>=PI2)IF_ETheta-=PI2;
        while (IF_ETheta<0)IF_ETheta+=PI2;
        Espeed=IF_ESpeed;
        Etheta=IF_ETheta;
        if (IF_ESpeed>=IF_Target_Speed) {
            IF_Start_Step=1;
        }
    }
    else if (IF_Start_Step==1) {
        IF_ETheta+=IF_ESpeed*T_s;
        while (IF_ETheta>=PI2)IF_ETheta-=PI2;
        while (IF_ETheta<0)IF_ETheta+=PI2;
        IF_Weight++;
        Espeed=IF_ESpeed;
        Etheta=IF_ETheta;
        float smo_IF_theta_error=my_abs(IF_ETheta-smo_pll_est.Etheta_O);
        if (smo_IF_theta_error<0.05f || smo_IF_theta_error>PI2-0.05f) {

Speed_PIstate.AddUp=(Iq_PIstate.Set-(Speed_PIstate.Set-Speed_PIstate.Measure)*Speed_P_VAL)*(1.f/Speed_I_Ts_V
AL);

        Speed_PIstate.Output=Iq_PIstate.Set;
        IF_Start_Step=2;
    }
}
else {
    Espeed=smo_pll_est.Espeed_O;
    Etheta=smo_pll_est.Etheta_O;
}

}

else if (Observer==3) {
    /*
    * 执行一次 ST-SMO-PLL, 获取转子角度和速度, 注意 ST-SMO 低速性能差, 这里先使用磁链观测器把速度提上来再切换到
    ST-SMO
    */

```

```

    */
    st_smo_pll_est.Ialpha_I=ClarkePark.clarke.Ialpha_O;
    st_smo_pll_est.Ibeta_I=ClarkePark.clarke.Ibeta_O;
    st_smo_pll_est.Valpha_I=Valpha_last;
    st_smo_pll_est.Vbeta_I=Vbeta_last;
    ST_SMO_PLL_update(&st_smo_pll_est);

    Espeed=st_smo_pll_est.Espeed_O;
    Etheta=st_smo_pll_est.Etheta_O;
}

/*
 * 每隔 10 次执行一次转速环(即 20khz / 10=2khz) , 获取 Q 电流环给定
 */
static int Speed_RunCount=0;
Speed_RunCount++;
if (Speed_RunCount>=10) {
    Speed_PIstate.Measure=Espeed;
    Speed_PI_update(&Speed_PIstate);
    Speed_RunCount=0;
}

/*
 * 执行一次 Park , 获取 dq 电流
 */
ClarkePark.park.Ialpha_I=ClarkePark.clarke.Ialpha_O;
ClarkePark.park.Ibeta_I=ClarkePark.clarke.Ibeta_O;
ClarkePark.park.Theta_I=Etheta;
Park_transform(&ClarkePark.park);

/*
 * 低通滤波滤除 id,iq 高频分量
 */
float id_lowpass = lowPass_filter_Id(ClarkePark.park.Id_O);
float iq_lowpass = lowPass_filter_Iq(ClarkePark.park.Iq_O);

/*
 * 执行一次 dq 电流环 , 获取 Udq 电压给定
 */
if (IF_Start_Step==0) {
    Id_PIstate.Set=0.2f;
}
else if (IF_Start_Step==1) {
    Id_PIstate.Set=0.2f;
}

```

```

}

else if (Id_PIstate.Set>=0.f)Id_PIstate.Set-=0.00001f;
else Id_PIstate.Set=0.f;

// Id_PIstate.Set=GetIdSet_Weak(Speed_PIstate.Set,Speed_PIstate.Measure);

Id_PIstate.Measure=id_lowpass;
Id_PI_update(&Id_PIstate);
if (IF_Start_Step==0) {
    if (IF_IqCurrent<IF_IqCurrentTarget)IF_IqCurrent+=IF_IqCurrentAcceleration*T_s;
    Iq_PIstate.Set=IF_IqCurrent;
}
else if (IF_Start_Step==1) {
    Iq_PIstate.Set=IF_IqCurrent*(IF_Weight_Total-IF_Weight)*(1.0f/IF_Weight_Total);
}

else Iq_PIstate.Set=Speed_PIstate.Output;
Iq_PIstate.Measure=iq_lowpass;
Iq_PI_update(&Iq_PIstate);

/*
 * 执行反park 前, 将上次设置的Ualpha 和Ubeta 记录下来, 用于下一次中断时的观测器预测
 */
Valpha_last=ClarkePark.ipark.Valpha_O;
Vbeta_last=ClarkePark.ipark.Vbeta_O;

/*
 * dq 电流解耦与反电动势前馈
 * 根据公式 :
 *  $ud=Rsid+Ld(di/dt)-weLqiq$ 
 *  $uq=Rsiq+Lq(di/dt)+we(Ldid+flux)$ 
 * 可知这里得到的dq 电流环输出ud,uq , 并不能准确反映id 或者iq 的大小,通常这里可以减去耦合项we*xxx
 */
static const uint8_t use_decoupling=0; // 是否使用前馈解耦
const float ud_decoupling=use_decoupling ? -Espeed*Ls*Iq_PIstate.Measure : 0;
const float uq_decoupling=use_decoupling ? Espeed*(Ls*Id_PIstate.Measure+flux) : use_decoupling;

/*
 * 执行一次反park , 获取Ualpha 和Ubeta
 */
ClarkePark.ipark.Vd_I=Id_PIstate.Output+ud_decoupling;
ClarkePark.ipark.Vq_I=Iq_PIstate.Output+uq_decoupling;
ClarkePark.ipark.Theta_I=Etheta;
IPark_transform(&ClarkePark.ipark);

/*
 * 对计算所得的矢量进行限幅(6.5V)

```

```

    */

float modulus;
arm_sqrt_f32(ClarkePark.ipark.Valpha_O*ClarkePark.ipark.Valpha_O
    +ClarkePark.ipark.Vbeta_O*ClarkePark.ipark.Vbeta_O,&modulus);
if (modulus>6.5f) {
    ClarkePark.ipark.Valpha_O *= 6.5f / modulus;
    ClarkePark.ipark.Vbeta_O*=6.5f/modulus;
}

/*
 * 执行一次 SVPWM , 更新计数值
 */
SVPWM_Calculate_Set(ClarkePark.ipark.Valpha_O,ClarkePark.ipark.Vbeta_O);

// 记录数据
recordRunningData();

/* USER CODE END ADC1_2_IRQn 1 */
}

/**
 * @brief This function handles USB low priority interrupt remap.
 */
void USB_LP_IRQHandler(void)
{
    /* USER CODE BEGIN USB_LP_IRQn 0 */

    /* USER CODE END USB_LP_IRQn 0 */
    HAL_PCD_IRQHandler(&hpcd_USB_FS);
    /* USER CODE BEGIN USB_LP_IRQn 1 */

    /* USER CODE END USB_LP_IRQn 1 */
}

/* USER CODE BEGIN 1 */

/* USER CODE END 1 */

```